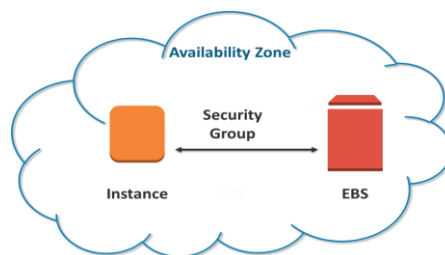


Rapid Elasticity using Amazon EBS, Amazon EFS, Amazon VPC

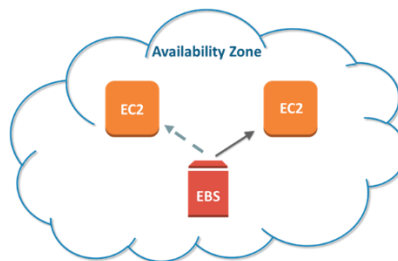
Amazon EBS

Elastic Block Store (EBS) is a **block storage service** provided by Amazon Web Services (AWS) for use with **Amazon Elastic Compute Cloud (EC2) instances**. It offers durable, high-performance **block-level storage** volumes that can be **attached to EC2 instances**, providing **persistent storage for data**.

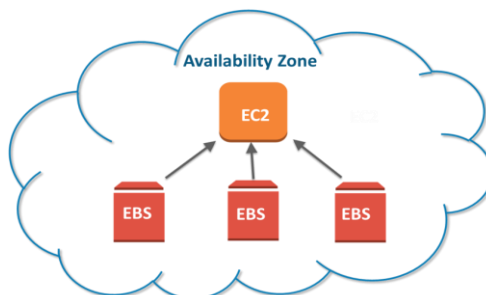
- EBS is a **logical volume associated with the EC2 instances**
- This type of storage is used when the data needs to be **accessed quickly** and is **required for a long time**
- **Lifetime of the EBS is not dependent on the EC2 instance**
- EBS volume and instance must be **located in the same Availability Zone**



- It can be **detached and attached between the instances in the same Availability Zone**



- **Any number of EBS Volumes can be attached with an EC2 instance**



Features of EBS:

Persistent Storage: EBS volumes **persist independently** from the life of an EC2 instance. This means that even if you **stop or terminate an EC2 instance**, the data stored on its attached EBS volumes remains intact.

High Performance: EBS volumes provide **consistent and low-latency performance**, making them suitable for a wide range of applications, from small databases to high-performance computing workloads.

Scalability: EBS volumes can be **easily resized to meet changing storage requirements without any downtime**. You can increase or decrease the size of an EBS volume and adjust its performance characteristics as needed.

Data Encryption: EBS volumes can be encrypted using AWS Key Management Service (KMS) **keys to protect data at rest**. This helps ensure the confidentiality and integrity of data stored on EBS volumes.

Snapshot Backups: EBS volumes can be backed up using snapshots, which are **point-in-time** copies of the volume's data stored in Amazon Simple Storage Service (S3). Snapshots are incremental, meaning that **only the changes made since the last snapshot are stored**, reducing storage costs and improving backup efficiency.

High Availability and Durability: EBS volumes are designed for high availability and durability. They are replicated within a single Availability Zone (AZ) to protect against component failures. Additionally, you can create snapshots and copy them across multiple AZs or regions for added redundancy.

Types of EBS Volumes:

General Purpose (SSD): Also known as gp2 volumes, these provide a balance of price and performance for a wide variety of workloads. They offer low-latency performance and are suitable for most applications.

Provisioned IOPS (SSD): Also known as **io1 volumes**, these are designed for I/O-intensive workloads that require consistent and predictable performance. You can specify the desired number of IOPS (Input/Output Operations Per Second) when creating io1 volumes.

Throughput Optimized (HDD): Also known as st1 volumes, these are designed for frequently accessed, **throughput-intensive workloads**, such as **big data processing and data warehousing**.

Cold HDD: Also known as sc1 volumes, these provide low-cost storage for infrequently accessed data. They are suitable for scenarios where cost optimization is a priority and performance requirements are moderate.

Magnetic (Standard): Also known as magnetic volumes, these offer the lowest cost per gigabyte of all EBS volume types. They are suitable for workloads with low-performance requirements or where cost is the primary consideration.

Rapid elasticity means that you can quickly scale up or down the amount of storage space you're using. For example, if you suddenly need more storage space because your application is growing, you can easily increase the capacity. Conversely, if you find that you're using more storage space than necessary, you can reduce it just as easily.

Elastic Block Storage (EBS) is like having a virtual hard drive for your computer in the cloud.

Imagine you have a computer where you can store all your files and data.

In the cloud, your computer is virtual, but you still need a place to store your stuff. That's where EBS comes in.

The "elastic" part of EBS means that you can easily adjust the size of your storage volumes as needed. If you need more space, you can simply increase the size of your EBS volume. And if you no longer need as much space, you can decrease it. This flexibility makes it easy to scale your storage up or down depending on your needs.

First we understand the terminology of vertical and horizontal scaling, and then we'll dive into Amazon Elastic Block Store (EBS) in AWS.

1. Vertical Scaling:

Vertical scaling, also known as scaling up or scaling vertically, involves increasing the capacity of a single server or instance by adding more resources such as CPU, memory, or storage. In this approach, you typically upgrade the existing hardware components of the server to handle increased workload or resource requirements. For example, you might add more RAM to a server to improve its performance.

Vertical scaling has its limitations, as there's only so much you can upgrade a single server before reaching its maximum capacity. It also introduces potential points of failure, as all resources are concentrated in a single instance.

2. Horizontal Scaling:

Horizontal scaling, also known as scaling out or scaling horizontally, involves adding more servers or instances to distribute the workload across multiple machines. Instead of upgrading a single server, you add more servers to your infrastructure to handle increased demand. Each server operates independently and can share the workload with other instances.

Horizontal scaling offers greater scalability compared to vertical scaling, as it allows you to add more resources as needed without being limited by the capacity of a single server. It also provides redundancy and fault tolerance, as multiple instances can continue to operate even if one or more servers fail.

Key features and benefits of Amazon EBS:

1. Elasticity: With Amazon EBS, you can easily scale your storage capacity up or down based on your application's needs. You can create EBS volumes of different sizes and types (such as General Purpose SSD, Provisioned IOPS SSD, and Throughput Optimized HDD) and attach them to your EC2 instances as needed.

2. Durability and Availability: Data stored on EBS volumes is replicated within a single Availability Zone (AZ) to protect against component failure. You can also create snapshots of your EBS volumes to back up your data and restore it in case of data loss or corruption.

3. Performance: Amazon EBS offers different types of storage volumes optimized for different performance requirements. For example, General Purpose SSD volumes provide a balance of price and performance for a wide range of workloads, while Provisioned IOPS SSD volumes offer predictable performance for I/O-intensive applications.

4. Snapshots and Backup: You can create point-in-time snapshots of your EBS volumes, which are stored in Amazon S3. These snapshots can be used to back up your data, migrate volumes between Availability Zones, or create new volumes.

Create and configure storage services using Amazon EBS

Amazon EBS (Elastic Block Store): Amazon EBS is a high-performance block storage service designed for use with EC2 instances. It provides persistent storage for virtual machines, meaning data remains available even after the instance is stopped or terminated.

EBS volumes can be attached to EC2 instances and support features like snapshots, encryption, and scalability. Scale Up and Scale Down.

Scale Up /Scale Down (Vertical Scaling):

Definition: Scale up, also known as vertical scaling, involves increasing the resources (such as CPU, RAM, or disk space) of an existing single server or virtual machine to handle increased workload or performance requirements.

EBS Equivalent: increasing the size or performance specifications of an existing EBS volume

Procedure: increasing/decreasing of CPU and RAM

Step 1. - Navigate to the EC2 dashboard.

Step 2. - Stop the EC2 instance associated with the EBS volume.

Step 3. - Click "Actions" and then—instance settings—change the instance type"Modify Volume".

Step 4. -Increase/ decrease the size or change the volume type to a higher performance specification.

Step 5. - Click "Modify" to apply the changes.

Step 6: Now the start the machine you see with increase /decrease of size and performance

Attaching and Detaching Volumes with in same region of EC2

Step 1. Launching an Instance with Additional Volumes:

- Start by launching an instance from the EC2 dashboard. Choose your desired AMI (Amazon Machine Image), instance type, and other configurations.

- In the "Add Storage" step, you can add additional volumes. Specify the size and other attributes for these volumes.

Step 2. Detaching a Volume from an Instance:

- Navigate to the EC2 and select "Volumes" under EBS section from the left-hand menu.
- Locate the volume you want to detach from the instance.
- Select the volume, then choose "Actions" -> "Detach Volume".
- Confirm the detachment by clicking "Yes, Detach".

Step 3. Attaching a Volume to Another Instance:

- After detaching the volume, it will become available.
- Select the detached volume from the Volumes list.
- Choose "Actions" -> "Attach Volume".
- Select the instance you want to attach the volume to
- specify the device name (e.g., /dev/sdf), and then click "Attach".
- Once attached, the volume will appear as an additional disk on the chosen instance.

Step 4: Attach and Mount EBS Volume to Linux EC2

Attach the volume via AWS Console.

- Connect to EC2 via SSH.
- Check available disks (lsblk).

Here you see root volume and attached (eg:-xvdf)

```
lsblk -fs # Check filesystem type
```

Create a partition using fdisk:

```
fdisk /dev/xvdf
```

- o Press m → Show help menu
- o Press n → Create a new partition
- o Press p → Create a primary partition
- o Press Enter twice → Use default partition size

o Press w → Write changes and exit

Partprobe # Inform the kernel about partition changes

Verify the new partition:

Lsblk -fs

Format the Disk Partition

mkfs.xfs /dev/xvdf1 # For XFS

Verify the filesystem:

lsblk -fs

Mount the Partition

Steps:

Create a directory for mounting:

1. mkdir /mnt/archana

Mount the partition:

2. mount /dev/xvdf1 /mnt/archana

Verify the mount:

3. df -h

Make the Mount Persistent

Steps:

Edit /etc/fstab to add the following line:

nano /etc/fstab

Add this line at the end:

<device> <mount_point> <filesystem_type> defaults 0 0

Now you see your volume is persistent

Check it by detaching the volume from instance and attaching it to another instance...

Note :- you need to create the directory and mount it ...to retrieve the files

Attaching Volumes across regions of EC2 using the snapshot

To create a snapshot of an EBS volume and attach it to an instance in another region in AWS, you'll need to follow these steps:

Step 1. Create a Snapshot:

- a. Sign in to the AWS Management Console.
- b. Navigate to the EC2 Dashboard.
- c. Click on "Volumes" in the left-hand navigation pane.
- d. Select the EBS volume you want to create a snapshot of.
- e. Click on the "Actions" dropdown menu above the volume list and select "Create Snapshot."
- f. Provide a name and description for the snapshot.
- g. Click on the "Create Snapshot" button to initiate the snapshot creation process.

Step 2. Copy the Snapshot to Another Region:

- a. Once the snapshot is created, go to the "Snapshots" section in the EC2 Dashboard.
- b. Select the snapshot you just created.
- c. Click on the "Actions" dropdown menu above the snapshot list and select "Copy Snapshot."
- d. Choose the destination region where you want to copy the snapshot.
- e. Click on the "Copy Snapshot" button to initiate the copy process. This may take some time depending on the size of the snapshot and the network speed.

Step 3. Monitor the Snapshot Copy Progress:

a. You can monitor the progress of the snapshot copy by navigating to the "Snapshots" section in the EC2 Dashboard of the source region.

b. Look for the snapshot you copied and check its status. It will change from "pending" to "completed" once the copy process is finished.

Step 4. Switch to the Destination Region:

a. Use the region selector in the top-right corner of the AWS Management Console to switch to the destination region where you copied the snapshot.

Step 5. Create a Volume from the Snapshot:

- a. In the EC2 Dashboard of the destination region, go to the "Snapshots" section.
- b. Find the snapshot you copied from the source region.
- c. Click on the snapshot, then click on the "Actions" dropdown menu and select "Create Volume."
- d. Configure the volume settings, such as volume type, size, and availability zone.
- e. Click on the "Create Volume" button to create the volume from the snapshot.

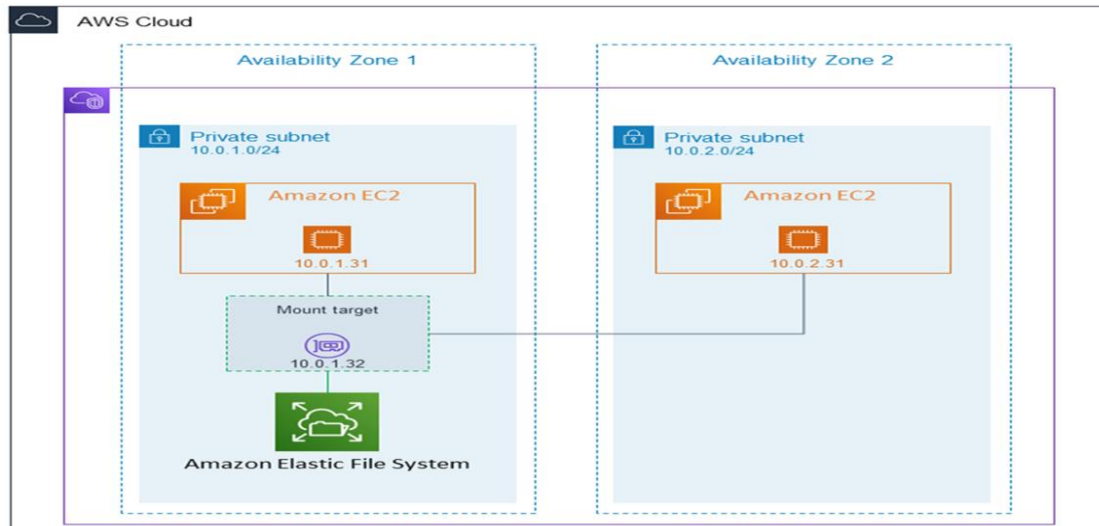
Step 6. Attach the Volume to an Instance:

- a. Once the volume is created, navigate to the "Volumes" section in the EC2 Dashboard.
- b. Find the newly created volume and select it.
- c. Click on the "Actions" dropdown menu and select "Attach Volume."
- d. Choose the EC2 instance to which you want to attach the volume and specify the device name.
- e. Click on the "Attach" button to attach the volume to the instance.

We have created a snapshot of an EBS volume, copied it to another region, created a volume from the snapshot, and attached it to an instance in the destination region.

Elastic File Storage

Amazon Elastic File System (EFS) is a scalable and fully managed file storage service provided by Amazon Web Services (AWS). It allows you to create and configure file systems that can be accessed concurrently from multiple EC2 instances, providing a highly available and scalable storage solution for your applications and workloads.



Amazon EFS supports for only Amazon Linux instance:

Step 1: Launching EC2 Instances

1. Sign in to AWS Management Console.
2. Go to the EC2 dashboard.
3. Click on "Launch Instance".
4. Configure instance details:
 - Instance Name: EFS-
5. Choose an Amazon Machine Image (AMI) for Linux.
6. Select instance type: t2.micro.
 - Key Pair: EFS
 - Network: Choose Subnet-1a
 - Security Group: Create a new security group (SG) and add NFS and allow from anywhere.
7. Launch the instance.

8. Repeat the above steps to launch another instance named EFS-2 in Subnet-1b with the same configuration.

Step 2: Creating an EFS File System

1. Go to the EFS service in the AWS Management Console.

2. Click on "Create file system".

3. Specify details:

- Name: Optional
- VPC: Default
- Enable region button.

Click on "Next".

4. In the Network settings:

- Delete all security groups.
- Select the newly created security group (NFS).

Click on "Next" and review the configuration.

Click on "Create file system".

Step 3: Accessing the two EC2 instances named EFS-1 & EFS -2 in two different PowerShell sessions and performing the specified tasks:

Accessing EFS-1 Instances in Two Different PowerShell Sessions:

1. Open Two PowerShell Sessions:

- Open two separate PowerShell windows or tabs on your local machine.

For each Instance:

2. SSH into the Instance:

- Use the SSH command to connect to the EFS-1 instance. Replace `[instance-public-ip]` with the public IP address of the instance:

```
ssh -i [path-to-your-keypair.pem] ec2-user@[instance-public-ip]
```

3. Switch to Root User:

- Gain root access by executing the following command:

```
sudo su
```

4. Create a Directory:

- Make a directory named "efs" using the following command:

```
mkdir efs
```

5. Install Amazon EFS Utilities:

- Use yum package manager to install the Amazon EFS utilities:

```
yum install -y amazon-efs-utils
```

6. List Files:

- Execute the following command to list files in the current directory:

```
ls
```

7. Verify Installation (Optional):

- Optionally, you can verify the installation of the Amazon EFS utilities by checking the version:

```
efs-utils --version
```

Repeat Steps 2-7 for Each PowerShell Session Corresponding to EFS-2 Instance.

By following these steps, you'll have accessed each EFS-1 instance in separate PowerShell sessions, switched to the root user, created a directory named "efs," installed the Amazon EFS utilities, and listed files in the directory. This setup allows you to configure and manage each instance individually as needed.

Step 4: Attaching EFS to EC2 Instances

1. Go to the EFS service in the AWS Management Console.
2. Click on the target EFS file system.

3. Click on the "Attach" button.
4. Choose "Mount via DNS" option.
5. Copy the displayed command.
6. SSH into each EC2 instance.
7. Paste and execute the copied command in the terminal to mount the EFS file system onto the instance.

Step 5: Verify and Test EFS

1. Change the directory to EFS on both instances.
2. Create a file on one instance.
3. Verify that the file automatically synchronizes and appears on the other instance.

By above steps, we can successfully create EC2 instances, configured them, created an EFS file system, and attached it to the instances in the same availability zone in the Mumbai region. Now, the instances can seamlessly access and share files stored in the EFS file system.

Virtual Private Network (VPC)

A Virtual Private Cloud (VPC) is a virtual network environment in the Amazon Web Services (AWS) cloud that closely resembles a traditional network that you might operate in your own data center.

It allows you to provision a logically isolated section of the AWS cloud where you can launch AWS resources in a virtual network that you define.

VPCs give you control over your network configuration, including IP address ranges, subnets, routing tables, network gateways, and security settings

CIDR Block:

CIDR is a method used to represent IP addresses and specify address ranges in a more flexible and efficient way than traditional IP address classes (Class A, B, and C).

- With CIDR, an IP address is followed by a slash ("/") and a number, known as the prefix length or subnet mask.

- The prefix length indicates the number of bits used for the network portion of the address.

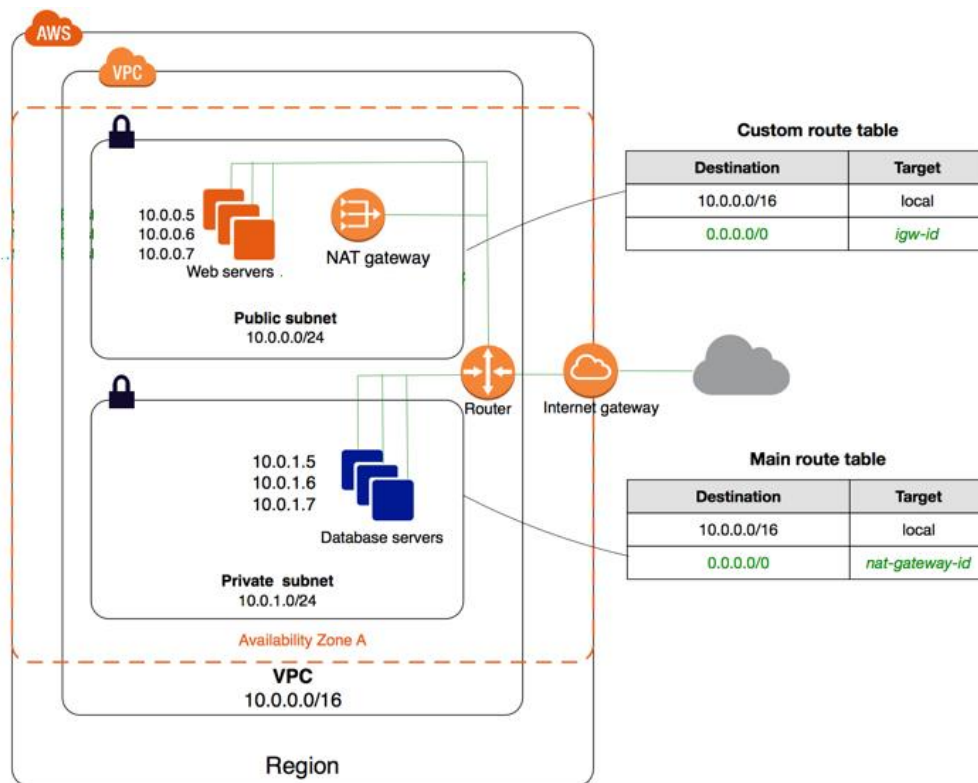
- For example, in the CIDR notation "192.168.1.0/24", the "/24" indicates that the first 24 bits represent the network portion of the address, leaving 8 bits for the host portion.

- For example, you might define a CIDR block of 10.0.0.0/16, which allows for up to 65,536 IP addresses.

Subnets:

- Subnets are segments of the VPC's IP address range where you can place groups of resources, such as EC2 instances, RDS databases, and Lambda functions.

- Each subnet resides in a specific Availability Zone (AZ) and has its own CIDR block, which is a subset of the VPC's CIDR block.



Steps to create a VPC (Virtual Private Cloud) network in AWS with one public server and another private server:

Step 1: Create a VPC

1. Sign in to the AWS Management Console.
2. Go to the VPC dashboard.
3. Click on "Create VPC."
4. Enter the following details:
 - Name tag: Provide a name for your VPC (e.g., MyVPC).
 - IPv4 CIDR block**: Define the IP address range for your VPC (e.g., 10.0.0.0/16).
5. Click on "Create."

Step 2: Create Subnets

1. In the VPC dashboard, click on "Subnets" in the left-hand navigation pane.
2. Click on "Create subnet."
3. Enter the following details:

- Name tag: Provide a name for your subnet (e.g., PublicSubnet).
- VPC: Select the VPC created in Step 1.
- Availability Zone: Choose an availability zone.
- IPv4 CIDR block: Define the IP address range for the subnet (e.g., 10.0.1.0/24 for a public subnet).

4. Click on "Create."

Note: Every subnet will be by default private.

We want to make one subnet as public.

To make subnet public its two step process.

Step 1: we need to enable public IP

Select the subnet (10.0.1.0/24) --> Actions --> Modify Auto Assigning IP Settings

---> Enable Auto Assigning public IPV4 Address -- Save

(From now, public IP will be assigned to the machines in this Subnet)

Repeat the above steps to create another subnet for the private server (e.g., PrivateSubnet with an IP address range like 10.0.2.0/24).

Step 3: Create Internet Gateway (IGW)

1. In the VPC dashboard, click on "Internet Gateways" in the left-hand navigation pane.
2. Click on "Create internet gateway."
3. Attach the internet gateway to your VPC.

Step 4: Update Route Table for Public Subnet

1. In the VPC dashboard, click on "Route Tables" in the left-hand navigation pane.

Now, we need to connect Route table to Subnet.

Select the route table (InternetRT) ---> Subnet Associations tab ---> Edit Subnet Associations
---> Select the subnet (10.0.1.0/24) -- Save

2. Select the route table associated with the public subnet.

3. Click on the "Routes" tab and then "Edit routes."
4. Add a route with destination `0.0.0.0/0` and target the internet gateway created in Step 3.
5. Click on "Save routes."

Step 5: Launch Instances

1. Go to the EC2 dashboard.
2. Launch a new EC2 instance for the public server:
 - Select the appropriate AMI (Amazon Machine Image).
 - Choose the public subnet created in Step 2.
 - Configure security groups to allow inbound traffic on port 80 (HTTP) or other necessary ports.
3. Launch another EC2 instance for the private server:
 - Select the appropriate AMI.
 - Choose the private subnet created in Step 2.
 - Ensure the security group only allows necessary inbound traffic from the public server or other trusted sources.

Step 6: Access Configuration

- The public server will have a public IP address and can be accessed directly over the internet.
- The private server will not have a public IP address and can only be accessed from within the VPC or through a VPN connection.